

Standard Operating Procedure: Deadlock Auditing Automation on Onprem SQL Server (Verified on SQL Server 2022)

Document ID: SOP-DBA-V-1.0

Comprehensive Standard Operating Procedure (SOP) for setting up, automating, and maintaining deadlock auditing in SQL Server.

Objective: To establish a permanent, automated mechanism for capturing, parsing, and storing SQL Server deadlock graphs into a query able historical table for trend analysis and troubleshooting.

1. Prerequisites & Environment Check

Before proceeding, ensure the following requirements are met:

- **Permissions:** You must have ALTER ANY EVENT SESSION and CONTROL SERVER permissions, as well as db_owner rights on the database hosting the historical data (e.g., master or a dedicated DBA_Admin database).
- **Storage Path:** The directory D:\MSSQL\DeadLocks\ must exist on the underlying Windows file system.
- **Service Account Permissions:** The SQL Server Engine and SQL Server Agent service accounts must have **Modify** permissions on the D:\MSSQL\DeadLocks\ folder.

2. Step-by-Step Implementation

Step 2.1: Create the Historical Storage Table

Create a central repository to store parsed deadlock information. While the original script references master.dbo.DeadlockHistory, it is highly recommended to place this in a dedicated administration database if available.

```
USE [master];
GO
IF OBJECT_ID('dbo.DeadlockHistory', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.DeadlockHistory (
        RecordID INT IDENTITY(1,1) NOT NULL,
        DeadlockTime DATETIME2 NOT NULL,
        DatabaseName NVARCHAR(200) NULL,
        ObjectName NVARCHAR(500) NULL,
        -- Victim Info
        Victim_SPID INT NULL,
        Victim_Login NVARCHAR(200) NULL,
        Victim_Host NVARCHAR(200) NULL,
        Victim_Application NVARCHAR(200) NULL,
        Victim_SQL NVARCHAR(MAX) NULL,
        -- Owner/Blocker Info
        Owner_SPID INT NULL,
        Owner_Login NVARCHAR(200) NULL,
        Owner_Host NVARCHAR(200) NULL,
        Owner_Application NVARCHAR(200) NULL,
        Owner_SQL NVARCHAR(MAX) NULL,
        -- Raw Graph
        DeadlockGraph XML NOT NULL,
        CaptureDate DATETIME2 DEFAULT SYSUTCDATETIME(),

        CONSTRAINT PK_DeadlockHistory PRIMARY KEY CLUSTERED (RecordID)
    );
    CREATE NONCLUSTERED INDEX IX_DeadlockHistory_DeadlockTime
    ON dbo.DeadlockHistory(DeadlockTime);
END;
```

Step 2.2: Deploy the Extended Events Session

Deploy the Extended Event to target the local disk drive.

```
IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name = 'CaptureDeadlocks')
    DROP EVENT SESSION [CaptureDeadlocks] ON SERVER;
CREATE EVENT SESSION [CaptureDeadlocks] ON SERVER
ADD EVENT sqlserver.xml_deadlock_report
ADD TARGET package0.event_file(
    SET filename=N'D:\MSSQL\DeadLocks\CaptureDeadlocks.xel',
    max_file_size=(50),
    max_rollover_files=(5)
)
WITH (
    MAX_MEMORY=4096 KB,
    EVENT_RETENTION_MODE=ALLOW_SINGLE_EVENT_LOSS,
    MAX_DISPATCH_LATENCY=30 SECONDS,
    MAX_EVENT_SIZE=0 KB,
    MEMORY_PARTITION_MODE=NONE,
    TRACK_CAUSALITY=OFF,
    STARTUP_STATE=ON -- Ensures it starts automatically with SQL Server
);
-- Start the session immediately
ALTER EVENT SESSION [CaptureDeadlocks] ON SERVER STATE = START;
```

Step 2.3: Deploy the Extraction Stored Procedure

This procedure handles extracting data safely from the active event file, sanitizing the file path, checking for duplicates, and logging to the table.

```
CREATE OR ALTER PROC USP_Deadlocks_Capture
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @FilePath NVARCHAR(4000);
    -- 1. Extract file path from active targets
    SELECT @FilePath = CAST(t.target_data AS XML).value('/EventFileTarget/File/@name'[1], 'nvarchar(4000)')
    FROM sys.dm_xe_sessions s
    JOIN sys.dm_xe_session_targets t ON s.address = t.event_session_address
    WHERE s.name = 'CaptureDeadlocks'
    AND t.target_name = 'event_file';
    -- 2. Guard Clause: Verify session is active
    IF @FilePath IS NULL
    BEGIN
        RAISERROR('Target file for Extended Event session "CaptureDeadlocks" could not be found. Ensure the session is running.',
        16, 1);
        RETURN;
    END
    -- 3. Standardize path format using wildcards
    IF @FilePath NOT LIKE '%*.xel'
    BEGIN
        IF @FilePath LIKE '%.xel'
            SET @FilePath = LEFT(@FilePath, LEN(@FilePath) - 4) + '*.xel';
        ELSE
            SET @FilePath = @FilePath + '*.xel';
    END
END
```

-- 4. Process and stage XEvent XML data

```
;WITH XEvents AS
(
    SELECT CAST(event_data AS XML) AS DeadlockXML,
           CAST(event_data AS XML).value('/event/@timestamp'[1], 'datetime2') AS DeadlockTime
    FROM sys.fn_xe_file_target_read_file(@FilePath, NULL, NULL, NULL)
),
Processes AS
(
    SELECT
        x.DeadlockTime,
        p.value('@id','varchar(100)') AS ProcessID,
        p.value('@spid','int') AS SPID,
        p.value('@loginname','nvarchar(200)') AS LoginName,
        p.value('@hostname','nvarchar(200)') AS HostName,
        p.value('@clientapp','nvarchar(200)') AS AppName,
        p.value('@currentdbname','nvarchar(200)') AS DBName,
        p.value('@sqlhandle','varbinary(64)') AS SqlHandle,
        p.value('inputbuf/text()')[1], 'nvarchar(max)') AS InputBuffer,
        x.DeadlockXML
    FROM XEvents x
    CROSS APPLY x.DeadlockXML.nodes('/event/data/value/deadlock/process-list/process') n(p)
),
Victim AS
(
    SELECT x.DeadlockTime, v.value('@id','varchar(100)') AS VictimID
    FROM XEvents x
    CROSS APPLY x.DeadlockXML.nodes('/event/data/value/deadlock/victim-list/victimProcess') n(v)
)
```

-- 5. Insert rows filtering out events already processed

```
INSERT INTO master.dbo.DeadlockHistory
(
    DeadlockTime, Victim_SPID, Victim_Login, Victim_Host, Victim_Application, Victim_SQL,
    Owner_SPID, Owner_Login, Owner_Host, Owner_Application, Owner_SQL,
    DatabaseName, ObjectName, DeadlockGraph
)
SELECT p1.DeadlockTime,
       p1.SPID, p1.LoginName, p1.HostName, p1.AppName, COALESCE(st1.text, p1.InputBuffer),
       p2.SPID, p2.LoginName, p2.HostName, p2.AppName, COALESCE(st2.text, p2.InputBuffer),
       p1.DBName,
       p1.DeadlockXML.value('/event/data/value/deadlock/resource-list/keylock/@objectname')[1], 'nvarchar(500)'),
       p1.DeadlockXML
FROM Victim v
JOIN Processes p1 ON p1.ProcessID = v.VictimID
JOIN Processes p2 ON p2.DeadlockTime = p1.DeadlockTime AND p2.ProcessID <> v.VictimID
OUTER APPLY sys.dm_exec_sql_text(p1.SqlHandle) st1
OUTER APPLY sys.dm_exec_sql_text(p2.SqlHandle) st2
WHERE NOT EXISTS (
    SELECT 1 FROM master.dbo.DeadlockHistory dh
    WHERE dh.DeadlockTime = p1.DeadlockTime
    AND dh.Victim_SPID = p1.SPID
);
END;
```

3. Automation via SQL Server Agent

To ensure historical preservation, create a SQL Server Agent job tasked with executing this extraction process at frequent intervals.

- **Job Name:** DBA_Capture_Deadlocks
- **Step 1 Name:** Execute USP_Deadlocks_Capture
- **Type:** Transact-SQL script (T-SQL)
- **Command:** EXEC master.dbo.USP_Deadlocks_Capture;
- **Schedule Type:** Recurring
- **Frequency:** Daily, occurring every 5 Minutes (Adjust down to 1 minute if deadlock occurrence is unusually high in your environment).

4. Maintenance and Data Retention

Because deadlocks output dense XML trees, the tracking table will grow over time.

Data Retention (Pruning) Policy

Add an optional second step to your SQL Server Agent Job or build a separate weekly job to prune historical entries past your data retention threshold (e.g., 90 days).

-- Retention Pruning Command

```
DELETE FROM master.dbo.DeadlockHistory  
WHERE DeadlockTime < DATEADD(DAY, -90, GETDATE());
```

5. Reporting & Troubleshooting Guide

Viewing Captured Deadlocks

To view the results sorted by the latest occurrence, run the query below.

Clicking the XML hyperlink in the DeadlockGraph column within SSMS will automatically open the graphical layout visualization.

```
SELECT  
    DeadlockTime,  
    DatabaseName,  
    ObjectName,  
    Victim_SPID,  
    Victim_SQL,  
    Owner_SPID,  
    Owner_SQL,  
    DeadlockGraph  
FROM master.dbo.DeadlockHistory  
ORDER BY DeadlockTime DESC;
```